

Problem 1.2

Solution:**(a)****Solution:**

Algorithm 1

```

1: for each  $a_i$  in  $A$  do
2:   for each  $b_i$  in  $B$  do
3:      $s = a_i + b_j$ 
4:     if  $s$  in  $C$  (using binary search) then                                 $\triangleright O(\log(n))$ 
5:       return True
6:     end if
7:   end for
8: end for
9: return False

```

Total time complexity is $O(|A||B|\log n)$.**(b)**

Algorithm 2

```

1: Define an array  $P$  of size  $n^{1.5}$  initialized to zero. For each element  $a \in A$ , set  $P[a] = 1$ .
2: Define an array  $Q$  of size  $n^{1.5}$  initialized to zero. For each element  $b \in B$ , set  $Q[b] = 1$ .
3: Compute the Discrete Fourier Transform of  $P$  and  $Q$  using Fast Fourier Transform,  $P' = \text{FFT}(P)$ ,  $Q' = \text{FFT}(Q)$ .                                 $\triangleright O(n^{1.5} \log n)$ 
4: Multiply the transformed arrays element-wise,  $R' = P' \cdot Q'$ .                                 $\triangleright O(n^{1.5})$ 
5: Compute the Inverse FFT to obtain the convolution result,  $R = \text{IFFT}(R')$ . Round the values in  $R$  to the nearest integers.
6: for each  $c$  in  $C$  do                                                                 $\triangleright O(n^{1.5})$ 
7:   if  $R[c] > 0$  then
8:     return True
9:   end if
10: end for
11: return False

```

Total time complexity is $O(n^{1.5} \log n)$.

(c)

Algorithm 3

- 1: Sort C .
 - 2: Define the maximum value as $max_val = n^{1.5}$.
 - 3: Choose the threshold parameter $0.5 < r < 1$ to distinguish between small and large intervals.
 - 4: Divide the set A and B into intervals of size $n^{1.5}$, and for each $a \in A$ and $b \in B$, assign it to its corresponding interval based on the value of $\lfloor a/n^{1.5} \rfloor$, $\lfloor b/n^{1.5} \rfloor$.
 - 5: For each interval of A and B with fewer than r elements, apply the algorithm from (a).
 - 6: Otherwise, apply the algorithm from (b).
-

The number of small or large intervals is at most $O(n/r)$.

The total time for large intervals is $O(n^{1.5} \log n) \times O(n/r) = O(n^{2.5}/r \log n)$

The total time for small intervals is $O(n \log n)$, each interval contributes at most $O(r^2 \log n)$ in the worst case. The total time for small intervals is $O(nr \log n)$ in the worst case.

Overall time complexity is $O((n^{2.5}/r + nr) \log n)$, which is supposed to be strictly smaller than $O(n^2)$, the choice of r must ensures that $n^{2.5}/r + nr < 2$, thus $0.5 < r < 1$.